DESIGN AND DEVELOPMENT OF AN INTELLIGENT REAL-TIME TRAFFIC CONGESTION PREDICTION SYSTEM USING MACHINE LEARNING

A ASSIGNMENT REPORT

A assignment Report submitted in partial fulfilment of the requirements

for the Course of

ITA0606: MACHINE LEARNING

to the award of the degree of

BACHELOR OF ENGINEERING IN

ELECTRONICS AND COMMUNICATION ENGINEERING

Submitted by

T.SRAVAN KUMAR (192412482)

Y.GOWTHAM REDDY (192412556)

GAUTHAM KRISHNA B.S (192525048)

Under the Supervision of

Dr BASKAR KASI

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

1. Problem Statement and Problem Formulation

Urban traffic congestion causes delays, inefficient road usage, excess fuel consumption and difficulty in traffic management. The proposed project develops a machine-learning system that predicts traffic congestion level from measurable traffic and contextual parameters.

Problem Formulation

Given a set of traffic observations containing vehicle count, average speed, road occupancy, hour, day and weather, the task is to learn a classification function that maps the input vector to one of three congestion classes: Low, Medium or High.

Mathematically, the model learns $f(X) \rightarrow y$, where $X = \{$vehicle count, average speed, occupancy, hour, day, weather$\}$ and $y \in \{$Low, Medium, High$\}$.

2. Objective and Expected Outcomes

- Develop a machine-learning model for traffic congestion classification.
- Preprocess numerical and categorical traffic variables using NumPy and Pandas.
- Analyze traffic patterns using statistical summaries and visualizations.
- Train and evaluate a Decision Tree classifier.
- Identify the traffic parameters that contribute most strongly to the prediction.
- Predict congestion for new traffic observations representing real-time input.

Expected Outcomes

- A working traffic congestion prediction simulation.
- Quantitative evaluation using accuracy, precision, recall and F1-score.
- Confusion matrix and Decision Tree visualization.
- Feature-importance evidence and graphical analysis.
- Example predictions for new traffic conditions.

3. Requirements, Constraints and Assumptions

Requirements

- Python 3.x environment, preferably Google Colab.
- NumPy, Pandas, Matplotlib and Scikit-learn.
- Traffic data containing numerical and categorical attributes.
- Train-test split and a supervised classification algorithm.

Constraints

- The demonstration uses a synthetic dataset rather than live traffic sensors.
- Prediction quality is limited by the simulated data-generation rules.
- The Decision Tree is restricted to a maximum depth of 5 in this experiment.
- Real incidents such as accidents, road closures and unusual events are not explicitly modelled.

Assumptions

- Vehicle count, speed and occupancy are measured consistently.

- Historical patterns provide useful information for congestion classification.
- Input categories can be encoded numerically without losing their practical meaning.
- The simulated labels are adequate for demonstrating the machine-learning workflow.

4. Application of Relevant Course Knowledge / Concepts

Concept	Application in Project
Supervised Learning	Congestion level is the target label.
Classification	Low, Medium and High are predicted classes.
Data Preprocessing	Categorical variables are label encoded and data are organized with Pandas.
Train-Test Split	80% of observations are used for training and 20% for testing.
Decision Tree	Entropy-based Decision Tree performs classification.
Feature Importance	Model importance values identify influential traffic parameters.
Evaluation Metrics	Accuracy, precision, recall and F1-score quantify performance.
Confusion Matrix	Shows correct and incorrect predictions by class.
Data Visualization	Graphs are used to interpret traffic distributions and relationships.

5. Design / Proposed Solution / Methodology

The proposed system follows a complete machine-learning pipeline: data generation/collection → preprocessing → exploratory analysis → feature/target separation → train-test split → Decision Tree training → prediction → evaluation → real-time-style inference.

System Architecture

Traffic Inputs → Data Preprocessing → Feature Encoding → Decision Tree Model → Congestion Prediction → Evaluation/Visualization → Traffic Management Decision Support

Methodology Steps

- Generate a 500-record synthetic traffic dataset using NumPy.
- Store and inspect the data using Pandas.
- Check missing values and encode categorical fields.
- Perform statistical analysis and visualize traffic patterns.
- Split the dataset into training and testing subsets.
- Train a Decision Tree Classifier using entropy and max_depth=5.
- Generate predictions on the test set.
- Calculate accuracy, precision, recall and F1-score.
- Plot the confusion matrix, feature importance and Decision Tree.
- Feed new traffic conditions to the trained model to demonstrate prediction.

6. Algorithm / Pseudocode / Flowchart

Algorithm

1. Start.

2. Generate or load traffic dataset.

3. Check and preprocess the dataset.

4. Encode categorical variables.

5. Separate features X and target y.

6. Split data into training and testing sets.

7. Train Decision Tree classifier.

8. Predict test-set classes.

9. Calculate evaluation metrics and confusion matrix.

10. Analyze feature importance.

11. Input new traffic data and predict congestion.

12. Display results and stop.

Pseudocode

```
BEGIN
   Generate traffic dataset
   Check missing values
   Encode Day, Weather and Congestion
   X ← traffic features
   y ← congestion class
   Split X, y into training and testing sets
   Train Decision Tree using entropy
   y_pred ← model.predict(X_test)
   Calculate accuracy, precision, recall and F1-score
   Display confusion matrix and feature importance
   Read new traffic input
   Predict congestion class
   Display prediction
END
```

7. Implementation / Source Code and Environment / Tools Used

Item	Details
Language	Python 3.x
Platform	Google Colab / Jupyter Notebook
Libraries	NumPy, Pandas, Matplotlib, Scikit-learn
Algorithm	DecisionTreeClassifier
Dataset	500-record synthetic traffic dataset generated with NumPy
Split	80% training / 20% testing
Tree criterion	Entropy
Maximum tree depth	5
Random seed	42

Source Code

```
import numpy as np

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score


np.random.seed(42)


n = 500

vehicle_count = np.random.randint(20, 201, n)

average_speed = np.random.randint(15, 81, n)

road_occupancy = np.random.uniform(10, 100, n)

hour = np.random.randint(0, 24, n)

day = np.random.choice(
```

```
    ["Monday","Tuesday","Wednesday","Thursday","Friday","Saturday","Sunday"], n)
weather = np.random.choice(["Clear","Rainy","Cloudy"], n)

congestion = []
for i in range(n):
    score = 0
    if vehicle_count[i] > 140: score += 2
    elif vehicle_count[i] > 90: score += 1
    if average_speed[i] < 30: score += 2
    elif average_speed[i] < 50: score += 1
    if road_occupancy[i] > 70: score += 2
    elif road_occupancy[i] > 45: score += 1
    if hour[i] in [7,8,9,17,18,19]: score += 1
    if weather[i] == "Rainy": score += 1
    congestion.append("High" if score >= 5 else "Medium" if score >= 3 else "Low")

df = pd.DataFrame({
    "Vehicle_Count": vehicle_count,
    "Average_Speed": average_speed,
    "Road_Occupancy": road_occupancy,
    "Hour": hour,
    "Day": day,
    "Weather": weather,
    "Congestion": congestion
})

day_encoder = LabelEncoder()
```

```
weather_encoder = LabelEncoder()
target_encoder = LabelEncoder()

df["Day"] = day_encoder.fit_transform(df["Day"])
df["Weather"] = weather_encoder.fit_transform(df["Weather"])
df["Congestion"] = target_encoder.fit_transform(df["Congestion"])

features = ["Vehicle_Count","Average_Speed","Road_Occupancy",
            "Hour","Day","Weather"]

X = df[features]
y = df["Congestion"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)

model = DecisionTreeClassifier(
    criterion="entropy", max_depth=5, random_state=42)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, average="weighted"))
print("Recall:", recall_score(y_test, y_pred, average="weighted"))
print("F1:", f1_score(y_test, y_pred, average="weighted"))
```

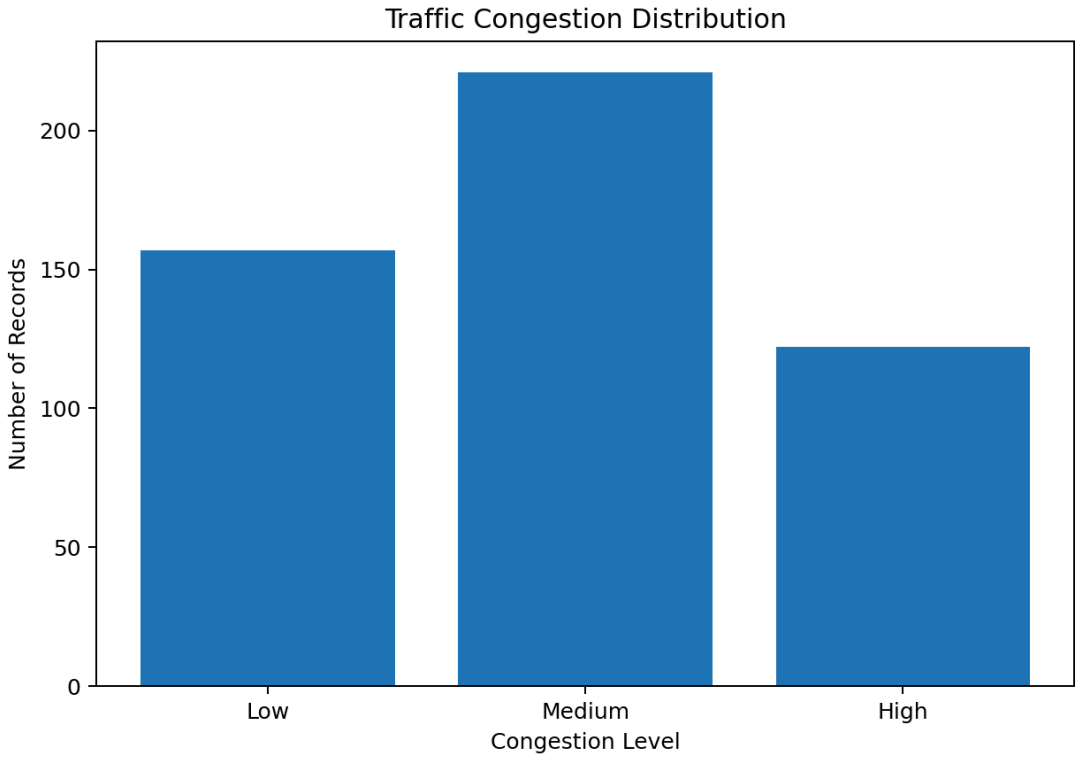
```
# New traffic prediction
new_traffic = pd.DataFrame({
    "Vehicle_Count":[165],
    "Average_Speed":[25],
    "Road_Occupancy":[82],
    "Hour":[18],
    "Day":[day_encoder.transform(["Monday"])[0]],
    "Weather":[weather_encoder.transform(["Rainy"])[0]]
})

prediction = model.predict(new_traffic)
print("Predicted congestion:",
      target_encoder.inverse_transform(prediction)[0])
```

8. Test Cases and Expected / Actual Results

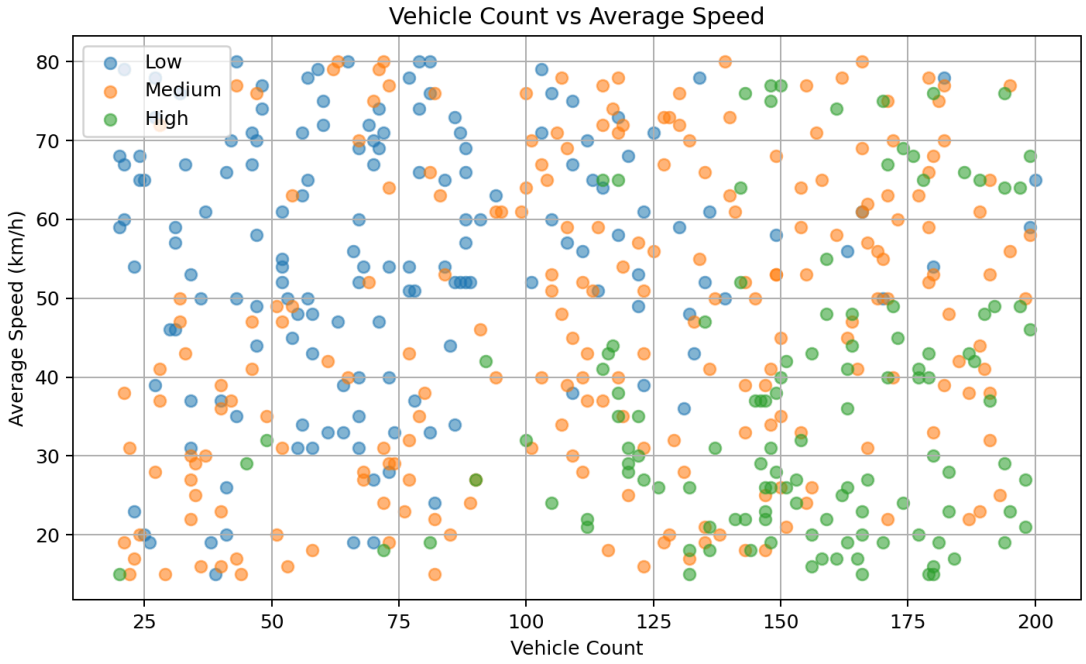
Test ID	Scenario	Input	Expected	Actual
TC01	Low traffic	45 vehicles, 65 km/h, 25% occupancy, 11:00, Clear	Low	Low
TC02	Medium traffic	100 vehicles, 45 km/h, 55% occupancy, 14:00, Cloudy	Medium	Medium
TC03	High traffic	175 vehicles, 20 km/h, 90% occupancy, 18:00, Rainy	High	High
TC04	Real-time sample	165 vehicles, 25 km/h, 82% occupancy, 18:00, Rainy	High	High

9. Execution Screenshots / Outputs

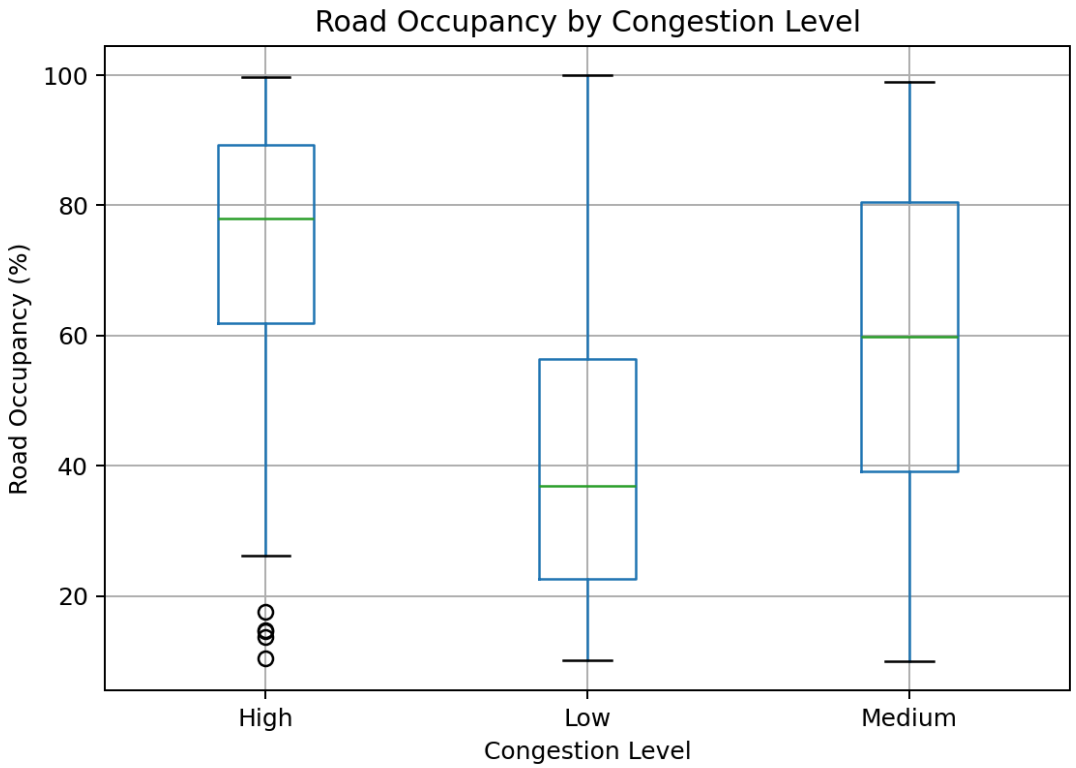
The following figures are generated from the same Python simulation and can be used as execution/output evidence. When submitting, run the notebook in Google Colab and optionally replace these figures with screenshots of the Colab output cells.



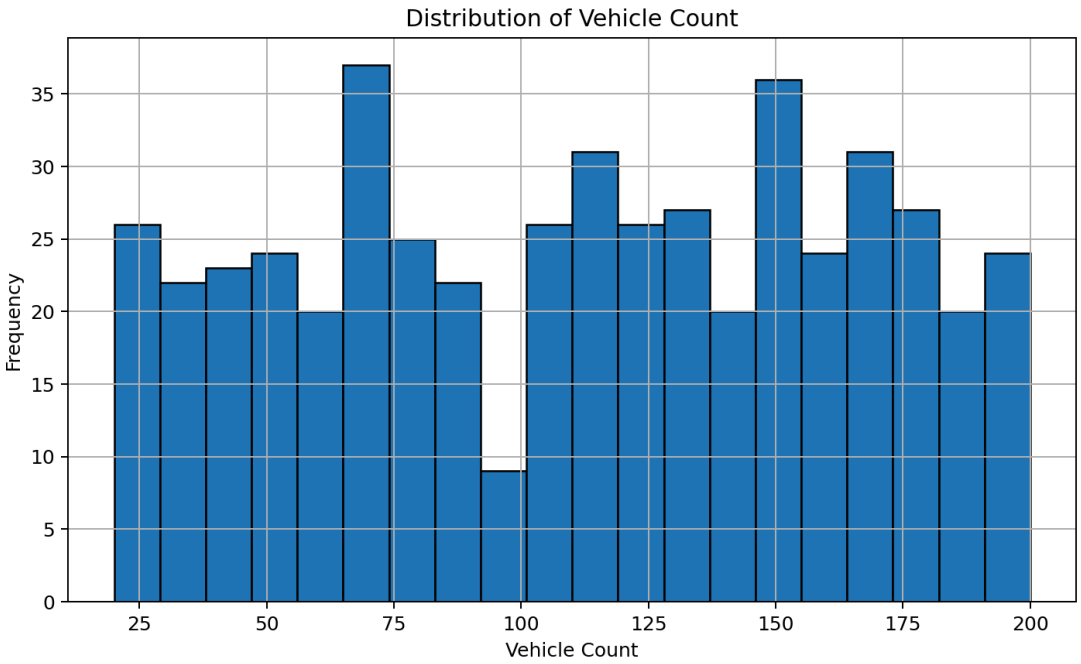
01 Congestion Distribution



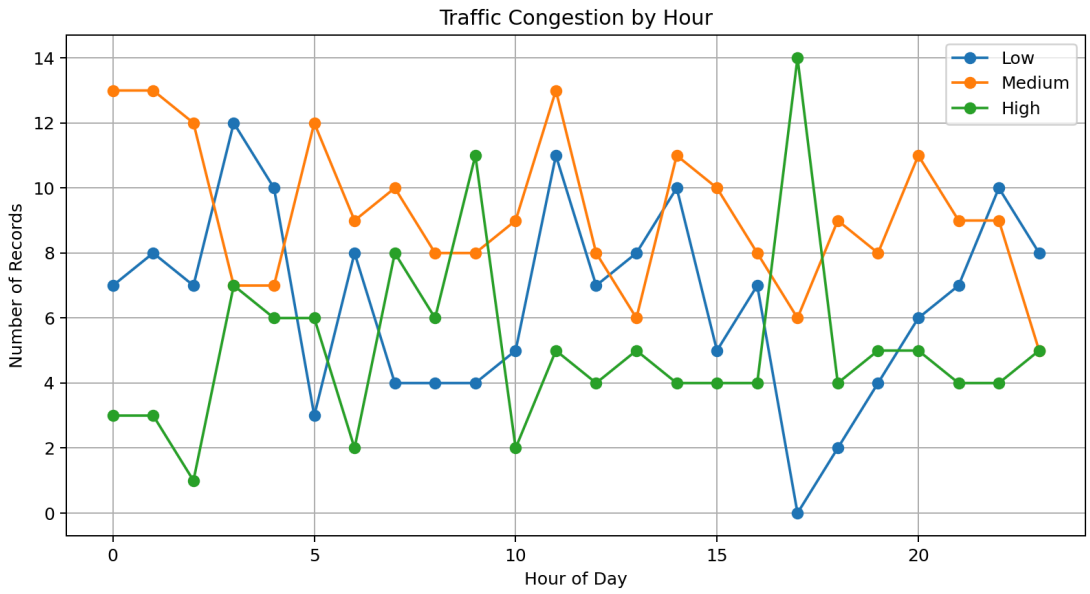
02 Vehicle Speed



03 Occupancy Boxplot



04 Vehicle Distribution



05 Hour Congestion

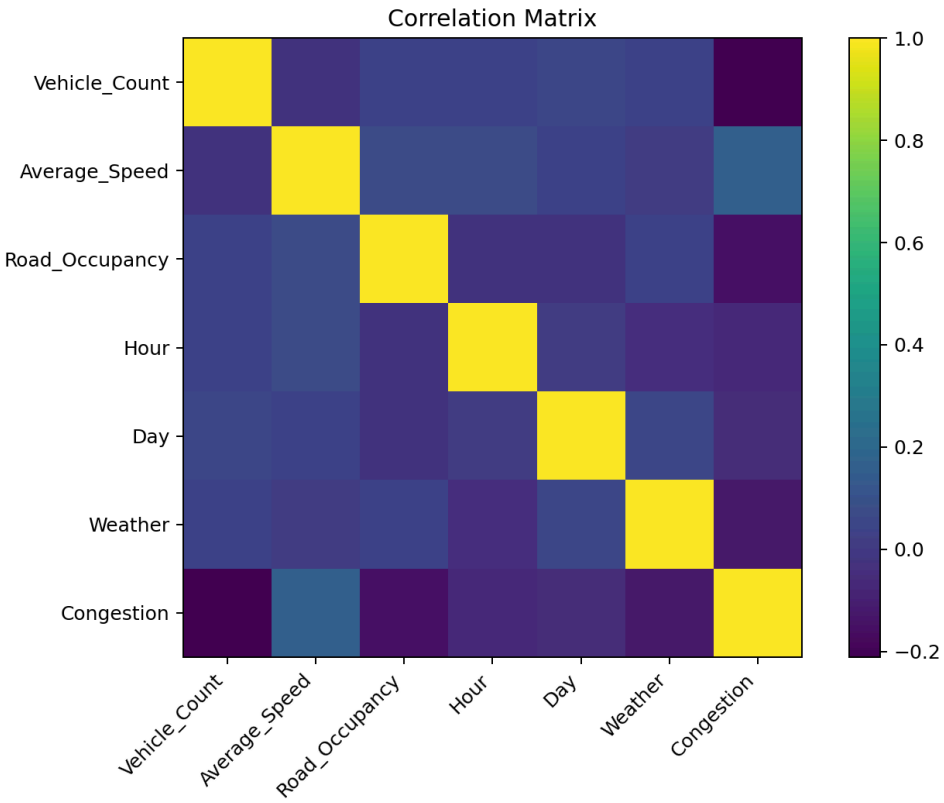
10. Results and Validation

Metric	Actual Result
Accuracy	80.00%
Weighted Precision	80.39%
Weighted Recall	80.00%
Weighted F1-Score	80.07%
Training records	400
Testing records	100

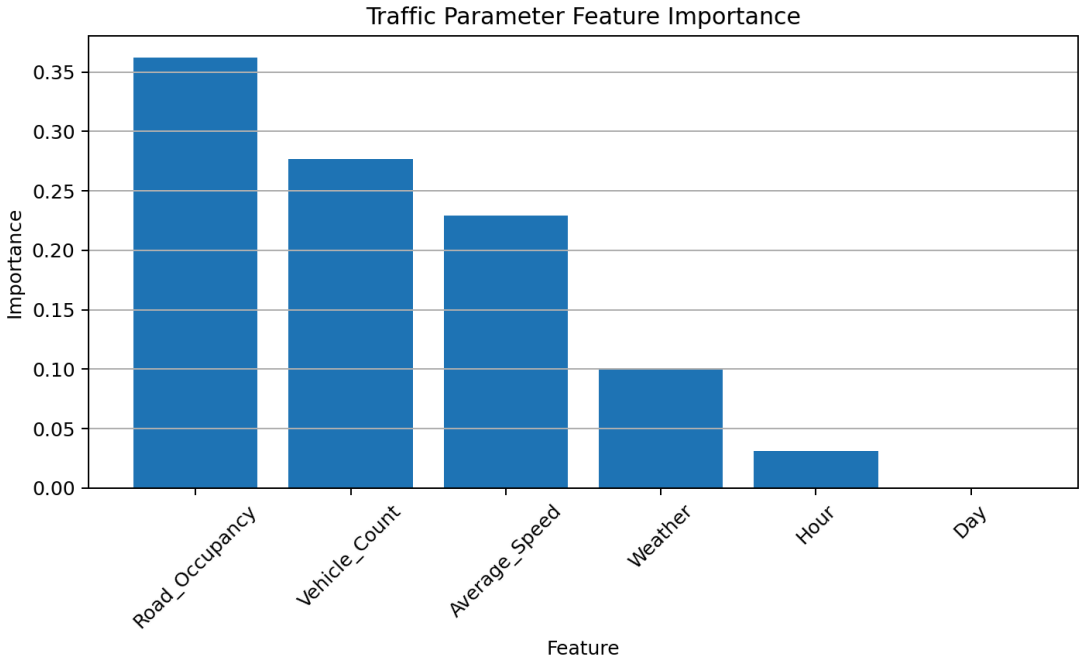
The evaluation results are calculated by executing the simulation with random seed 42, an 80:20 train-test split, entropy criterion and maximum tree depth of 5.

Confusion Matrix:

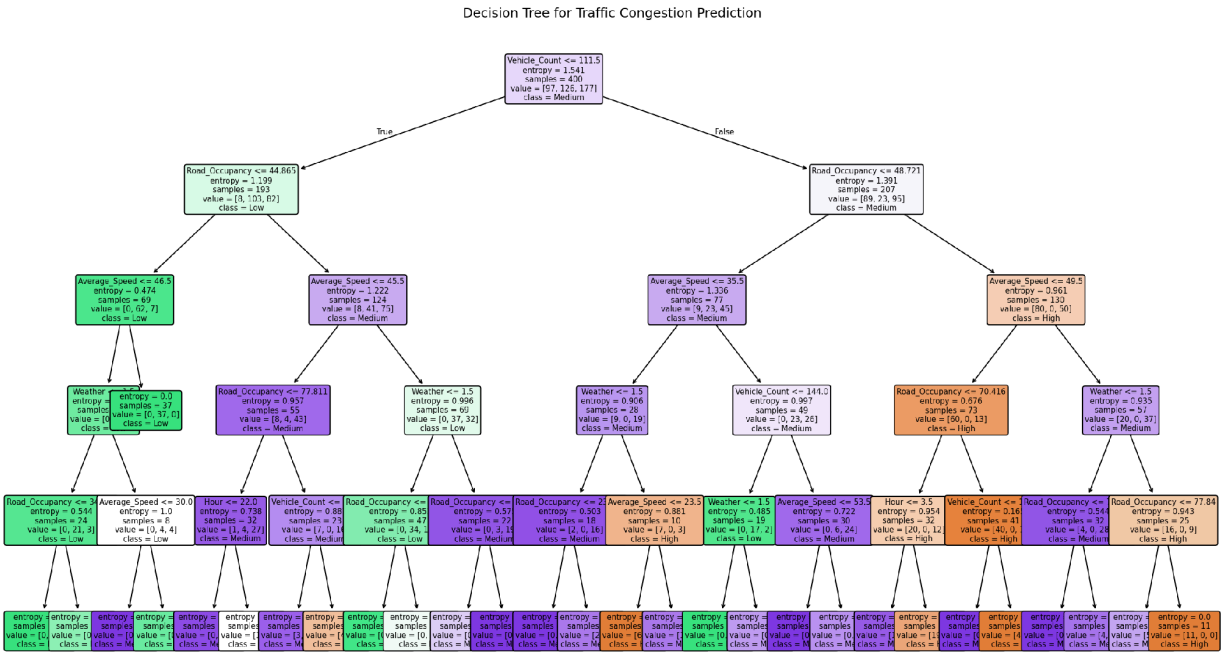
	Pred Low	Pred Medium	Pred High
Actual Low	20	0	5
Actual Medium	0	24	7
Actual High	3	5	36



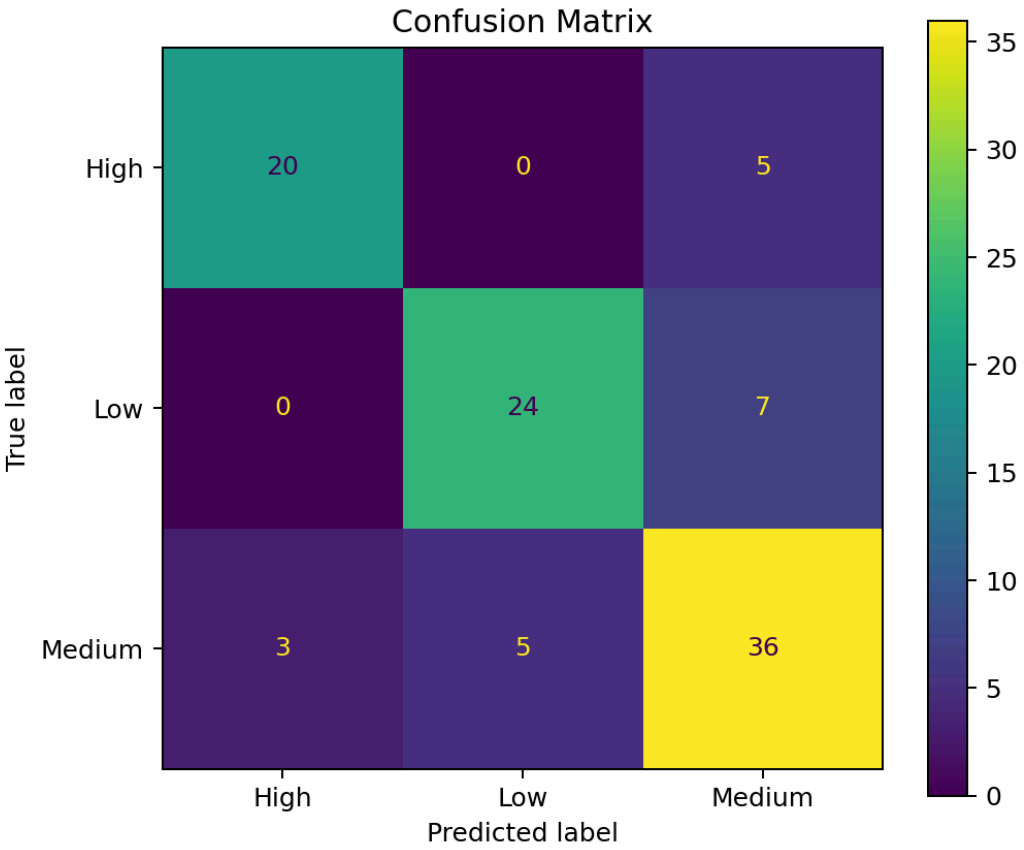
Correlation Matrix



Feature Importance



Decision Tree Visualization



Confusion Matrix

Feature Importance Table

Rank	Feature	Importance
1	Road_Occupancy	0.3622
2	Vehicle_Count	0.2766
3	Average_Speed	0.2295
4	Weather	0.1006
5	Hour	0.0311
6	Day	0.0000

11. Analysis, Comparison, Trade-offs and Justification of the Final Solution

The Decision Tree was selected because it is suitable for a multi-class classification problem and produces an interpretable set of decision rules. Its feature-importance output also makes it straightforward to explain which traffic parameters influence the prediction.

Approach	Strengths	Trade-offs
Decision Tree	Interpretable, fast, handles nonlinear rules, easy to visualize	Can overfit; performance depends on tree depth
Logistic Regression	Simple and computationally efficient	Less suitable for nonlinear traffic relationships
Random Forest	Usually more robust and less prone to single-tree overfitting	Less interpretable than one tree and more computationally expensive
Neural Network	Can model complex patterns	Requires more data, tuning and computational resources

For this academic prototype, interpretability and ease of implementation are more important than maximum model complexity. Therefore, the Decision Tree provides a justified final solution. For a production system, Random Forest, gradient boosting or time-series/deep-learning approaches should be benchmarked against the Decision Tree.

12. Broader Considerations / SDG Relevance

- SDG 9 – Industry, Innovation and Infrastructure: applies machine learning to intelligent transportation infrastructure.
- SDG 11 – Sustainable Cities and Communities: supports better traffic management and urban mobility.
- SDG 13 – Climate Action: reducing congestion and idling may help reduce unnecessary fuel use and emissions.
- Privacy: real deployments using cameras or location data should minimize personal-data collection and apply appropriate safeguards.
- Fairness and reliability: the model should be tested across different roads, times, weather conditions and traffic patterns to reduce dataset bias.

13. Conclusion, Limitations and Possible Improvements

Conclusion

The project demonstrates a complete machine-learning workflow for traffic congestion prediction. The Decision Tree classifier successfully maps traffic and contextual inputs to Low, Medium and High congestion classes and provides interpretable feature-importance and decision-tree outputs. The simulation also demonstrates how new traffic conditions can be classified as a real-time-style prediction.

Limitations

- The dataset is synthetic and does not represent every real-world traffic condition.
- Labels are generated from predefined rules, so high evaluation scores should not be interpreted as proof of real-world performance.
- Unexpected incidents such as accidents and road closures are not modelled.
- Temporal dependencies are simplified because the current prototype treats each observation independently.

Possible Improvements

- Replace synthetic data with real traffic sensor or public traffic datasets.
- Use continuous real-time streams from sensors or APIs.
- Compare Decision Tree with Random Forest, Gradient Boosting and time-series models.
- Perform cross-validation and hyperparameter tuning.
- Add accident, road-work, event and air-quality information.
- Deploy the model using a dashboard or edge/cloud architecture.

14. Individual Contribution of Group Members

Member	Contribution
T.SRAVAN KUMAR	Dataset generation, preprocessing and exploratory analysis,Report preparation, testing and documentation
Y.GOWTHAM REDDY	Decision Tree implementation and model evaluation
GAUTHAM KRISHNA B.S	Visualization, confusion matrix and feature-importance analysis

15. References

- Scikit-learn documentation – DecisionTreeClassifier and model evaluation metrics.
- NumPy documentation – numerical data generation and array operations.
- Pandas documentation – DataFrame creation, preprocessing and statistical analysis.

- Matplotlib documentation – data visualization and plotting.
- Google Colab – cloud-based Python notebook environment.
- Dataset: Synthetic traffic dataset generated within this project using NumPy; no external dataset was used for the simulation.

16. One-Page Individual Reflection

Working on the Design and Development of an Intelligent Real-Time Traffic Congestion Prediction System helped me understand how machine learning concepts can be applied to a practical smart-city problem. Through this assignment, I gained practical knowledge of data preprocessing using NumPy and Pandas, statistical analysis, visualization, Decision Tree classification, feature-importance analysis, and model evaluation using accuracy, precision, recall, F1-score and a confusion matrix.

One important limitation of the proposed system is that prediction quality depends strongly on the quality and representativeness of the traffic dataset. Real-world traffic conditions can change because of accidents, road construction, festivals, emergencies, unexpected weather or changes in driving behaviour. Therefore, predictions may contain uncertainty when the model receives conditions that are not sufficiently represented during training. The Decision Tree can also overfit if it is not properly tuned.

I also recognized the possibility of bias in the dataset. If traffic data is collected mainly from particular roads, time periods or weather conditions, the model may perform better for those situations and less accurately for other conditions. Regularly updating the dataset with diverse traffic information can help reduce this problem. From an ethical perspective, traffic prediction systems should use sensor and camera data responsibly, protect privacy and ensure that automated predictions are used as decision-support information rather than blindly replacing human judgement.

The system has good potential for scalability because the same machine-learning approach can be extended to multiple roads and traffic junctions. With real-time sensor or camera data, predictions can be continuously updated and used by traffic authorities for route management, dynamic signal timing and congestion reduction. More advanced models and cloud or edge-computing platforms could further improve real-time performance.

The project is relevant to SDG 9, SDG 11 and SDG 13 because it applies intelligent infrastructure to transportation, supports more efficient urban mobility and may reduce unnecessary idling and emissions. Overall, the assignment improved my understanding of the complete machine-learning workflow, from data preparation to model development, evaluation and interpretation. In future work, I would use a larger real-time dataset, continuously update the model, perform hyperparameter tuning, compare several algorithms and integrate live traffic sensors or APIs.